Chapter 1
Principles of Finite Precision Computation

Numerical precision is the very soul of science.
-SIR D'ARCY WENTWORTH THOMPSON, *On Growth and Form (1942)*

There will always be a small but steady demand for error-analysts to . . .
expose bad algorithms' big errors and, more important,
supplant bad algorithms with provably good ones.
-WILLIAM M. KAHAN, *Interval Arithmetic Options in the Proposed IEEE Floating Point Arithmetic Standard* (1980)

Since none of the numbers which we take out from logarithmic and
trigonometric tables admit of absolute precision,
but are all to a certain extent approximate only,
the results of all calculations performed
by the aid of these numbers can only be approximately true . . .
It may happen, that in special cases the
effect of the errors of the tables is so augmented that
we may be obliged to reject a method,
otherwise the best, and substitute another in its place.
-CARL FRIEDRICH GAUSS', *Theoria Motus* (1809)

Backward error analysis is no panacea;
it may explain errors but not excuse them.
-HEWLETT-PACKARD, *HP-15C Advanced Functions Handbook* (1982)

[1] Cited in Goldstine [461, 1977, p. 258].

This book is concerned with the effects of finite precision arithmetic on numerical algorithms', particularly those in numerical linear algebra. Central to any understanding of high-level algorithms is an appreciation of the basic concepts of finite precision arithmetic. This opening chapter briskly imparts the necessary background material. Various examples are used for illustration, some of them familiar (such as the quadratic equation) but several less well known. Common misconceptions and myths exposed during the chapter are highlighted towards the end, in §1.19.

This chapter has few prerequisites and few assumptions are made about the nature of the finite precision arithmetic (for example, the base, number of digits, or mode of rounding, or even whether it is floating point arithmetic). The second chapter deals in detail with the specifics of floating point arithmetic.

A word of warning: some of the examples from §1.12 onward are special ones chosen to illustrate particular phenomena. You may never see in practice the extremes of behaviour shown here. Let the examples show you what can happen, but do not let them destroy your confidence in finite precision arithmetic!

1.1. Notation and Background

We describe the notation used in the book and briefly set up definitions needed for this chapter.

Generally, we use

capital letters	$A, B, C \Delta, \Lambda$	for matrices,
subscripted lower case letters	$\alpha_{ij}, b_{ij}, c_{ij}, \delta_{ij}, \lambda_{ij}$	for matrix elements,
lower case letters	x, y, z, c, g, h	for vectors,
lower case Greek letters	$\alpha, \beta, \gamma, \phi, \rho$	for scalars,

following the widely used convention originally introduced by Householder [587, 1964].

The vector space of all real $m \times n$ matrices is denoted by $\mathbb{R}^{m \times n}$ and the vector space of real n-vectors by $\mathbb{R}^n$. Similarly, $\mathbb{C}^{m \times n}$ denotes the vector space of complex $m \times n$ matrices.

Algorithms are expressed using a pseudocode based on the MATLAB language [232, 1988], [735, 1992]. Comments begin with the % symbol.

Submatrices are specified with the colon notation, as used in MATLAB and Fortran 90: $A(p:q, r:s)$ denotes the submatrix of A formed by the intersection of rows p to q and columns r to s. As a special case, a lone colon as the row or column specifier means to take all entries in that row or column; thus $A(:,j)$ is the jth column of A and $A(i,:)$ the ith row. The values taken by an integer

[2]For the purposes of this book an algorithm is a MATLAB program; cf. Smale [924, 1990].

variable are also described using the colon notation: "$i = 1{:}n$" means the same as "$i = 1,2, \ldots, n$".

Evaluation of an expression in floating point arithmetic is denoted $fl(\cdot)$, and we assume that the basic arithmetic operations op=+,-,*,/ satisfy

$$fl(x \mathrm{\ op\ } y) = (x \mathrm{\ op\ } y)(1+\delta), \qquad |\delta| \le u. \tag{1.1}$$

Here, u is the *unit roundoff* (or machine precision), which is typically of order 10^{-8} or 10^{-16} in single and double precision computer arithmetic, respectively, and between 10^{-10} and 10^{-12} on pocket calculators. For more on floating point arithmetic see Chapter 2.

Computed quantities (and, in this chapter only, arbitrary approximations) wear a hat. Thus $\widehat{x}$ denotes the computed approximation to x.

Definitions are often (but not always) indicated by ":=" or "=:", with the colon next to the object being defined.

We make use of the floor and ceiling functions: $\lfloor x \rfloor$ is the largest integer less than or equal to x, and $\lceil x \rceil$ is the smallest integer greater than or equal to x.

The normal distribution with mean μ and variance σ^2 is denoted by $N(\mu, \sigma^2)$.

We measure the cost of algorithms in flops. A *flop* is an elementary floating point operation: +,-,/, or *. We normally state only the highest-order terms of flop counts. Thus, when we say that an algorithm for $n \times n$ matrices requires $2n^3/3$ flops, we really mean $2n^3/3 + O(n^2)$ flops.

Other definitions and notation are introduced when needed. Two top ics, however, do not fit comfortably into the main text and are described in Appendix B: the singular value decomposition (SVD) and M-matrices.

All our numerical experiments were carried out either in MATLAB 4.2 [735, 1992], sometimes in conjunction with the Symbolic Math Toolbox [204, 1993], or with the Salford Software/Numerical Algorithms Group FTN90[3] Fortran 90 compiler, Version 1.2 [888, 1993]. Whenever we say a computation was "done in Fortran 90" we are referring to the use of this compiler. All the results quoted were obtained on a 486DX workstation, unless otherwise stated, but many of the experiments were repeated on a Sun SPARCstation, using the NAGWare[4] FTN90 compiler [785, 1992]. Both machines use IEEE standard floating point arithmetic and the unit roundoff is $u = 2^{-53} \approx 1.1 \times 10^{-16}$ in MATLAB and in double precision in Fortran 90. (Strictly speaking, in Fortran 90 we should not use the terminology single and double precision but should refer to the appropriate KIND parameters; see, e.g., Metcalf and Reid [749, 1990, §2.6]. However, these terms are vivid and unambiguous in IEEE arithmetic, so we use them throughout the book.)

[3] FTN90 is a joint trademark of Salford Software Ltd. and The Numerical Algorithms Group Ltd.

[4] NAGWare is a trademark of The Numerical Algorithms Group Ltd.

1.2. Relative Error and Significant Digits

Let $\hat{x}$ be an approximation to a real number x. The most useful measures of the accuracy of $\hat{x}$ are its *absolute error*

$$E_{\mathrm{abs}}(\hat{x}) = |x - \hat{x}|,$$

and its *relative error*

$$E_{\mathrm{rel}}(\hat{x}) = \frac{|x - \hat{x}|}{|x|}$$

(which is undefined if $x = 0$). An equivalent definition of relative error is $E_{\mathrm{rel}}(\hat{x}) = |\rho|$, where $\hat{x} = x(1+\rho)$. Some authors omit the absolute values from these definitions. When the sign is important we will simply talk about "the error $x - \hat{x}$".

In scientific computation, where answers to problems can vary enormously in magnitude, it is usually the relative error that is of interest, because it is scale independent: scaling $x \rightarrow \alpha x$ and $\hat{x} \rightarrow \alpha\hat{x}$ leaves $E_{\mathrm{rel}}(\hat{x})$ unchanged.

Relative error is connected with the notion of correct significant digits (or correct significant figures). The significant digits in a number are the first nonzero digit and all succeeding digits. Thus 1.7320 has five significant digits, while 0.0491 has only three. What is meant by *correct* significant digits in a number that approximates another seems intuitively clear, but a precise definition is problematic, as we explain in a moment. First, note that for a number $\hat{x}$ with p significant digits there are only $p+1$ possible answers to the question "how many correct significant digits does $\hat{x}$ have?" (assuming $\hat{x}$ is not a constant such as 2.0 that is known exactly). Therefore the number of correct significant digits is a fairly crude measure of accuracy in comparison with the relative error. For example, in the following two cases $\hat{x}$ agrees with x to three but not four significant digits by any reasonable definition, yet the relative errors differ by a factor of about 44:

$$x = 1.00000, \quad \hat{x} = 1.00499, \quad E_{\mathrm{rel}}(\hat{x}) = 4.99 \times 10^{-3},$$
$$x = 9.00000, \quad \hat{x} = 8.99899, \quad E_{\mathrm{rel}}(\hat{x}) = 1.12 \times 10^{-4}.$$

Here is a possible definition of correct significant digits: *an approximation $\hat{x}$ to x has p correct significant digits if $\hat{x}$ and x round to the same number to p significant digits. Rounding* is the act of replacing a given number by the nearest p significant digit number, with some rule for breaking ties when there are two nearest. This definition of correct significant digits is mathematically elegant and agrees with intuition most of the time. But consider the numbers

$$x = 0.9949, \quad \hat{x} = 0.9951.$$

According to the definition $\hat{x}$ does not have two correct significant digits $(x \rightarrow 0.99, \hat{x} \rightarrow 1.0)$, but does have one and three correct significant digits!

or, if the data is itself the solution to another problem, it may be the result of errors in an earlier computation. The effects of errors in the data are generally easier to understand than the effects of rounding errors committed during a computation, because data errors can be analysed using perturbation theory for the problem at hand, while intermediate rounding errors require an analysis specific to the given method. This book contains perturbation theory for most of the problems considered, for example, in Chapters 7 (linear systems), 19 (the least squares problem), and 20 (underdetermined systems).

Analysing truncation errors, or discretization errors, is one of the major tasks of the numerical analyst. Many standard numerical methods (for example, the trapezium rule for quadrature, Euler's method for differential equations, and Newton's method for nonlinear equations) can be derived by taking finitely many terms of a Taylor series. The terms omitted constitute the truncation error, and for many methods the size of this error depends on a parameter (often called h, "the stepsize") whose appropriate value is a compromise between obtaining a small error and a fast computation.

Because the emphasis of this book is on finite precision computation, with virtually no mention of truncation errors, it would be easy for the reader to gain the impression that the study of numerical methods is dominated by the study of rounding errors. This is certainly not the case. Trefethen explains it well when he discusses how to define numerical analysis [1016, 1992]:

> Rounding errors and instability are important, and numerical analysts will always be the experts in these subjects and at pains to ensure that the unwary are not tripped up by them. But our central mission is to compute quantities that are typically uncomputable, from an analytic point of view, and to do it with lightning speed.

In this quotation "uncomputable" means that approximations are necessary, and thus Trefethen's point is that developing good approximations is a more fundamental task than analysing the effects of rounding errors on those approximations.

A possible way to avoid rounding and truncation errors (but not data errors) is to try to solve a problem using a symbolic manipulation package, such as Maple[5] [199, 1991] or Mathematica[6] [1109, 1991]. Indeed, we have used this approach to compute "exact answers" in some of our numerical experiments, While we acknowledge the value of symbolic manipulation as part of the toolkit of the scientific problem solver, we do not study it in this book.

[5]Maple is a registered trademark of Waterloo Maple Software.

[6]Mathematica is a registered trademark of Wolfram Research Inc.

A definition of correct significant digits that does not suffer from the latter anomaly states that *$\hat{x}$ agrees with x to p significant digits if $|x-\hat{x}|$ is less than half a unit in the pth significant digit of x.*. However, this definition implies that 0.123 and 0.127 agree to two significant digits, whereas many people would say that they agree to less than two significant digits.

In summary, while the number of correct significant digits provides a useful way in which to think about the accuracy of an approximation, the relative error is a more precise measure (and is base independent). Whenever we give an approximate answer to a problem we should aim to state an estimate or bound for the relative error.

When x and $\hat{x}$ are vectors the relative error is most often defined with a norm, as $\|x-\hat{x}\|/\|x\|$. For the commonly used norms $\|x\|_\infty := \max_i |x_i|$, $\|x\|_1 := \sum_i |x_i|$, and $\|x\|_2 := (x^T x)^{1/2}$, the inequality $\|x-\hat{x}\|/\|x\| \leq \frac{1}{2} \times 10^{-p}$ implies that components x_i with $|x_i| \approx \|x\|$ have about p correct significant decimal digits, but for the smaller components the inequality merely bounds the absolute error.

A relative error that puts the individual relative errors on an equal footing is the *componentwise relative error*

$$\max_i \frac{|x_i - \hat{x}_i|}{|x_i|},$$

which is widely used in error analysis and perturbation analysis (see Chapter 7, for example).

As an interesting aside we mention the "tablemaker's dilemma". Suppose you are tabulating the values of a transcendental function such as the sine function and a particular entry is evaluated as 0.124|500000000 correct to a few digits in the last place shown, where the vertical bar follows the final significant digit to be tabulated. Should the final significant digit be 4 or 5? The answer depends on whether there is a nonzero trailing digit and, in principle, we may never be able answer the question by computing only a finite number of digits.

1.3. Sources of Errors

There are three main sources of errors in numerical computation: rounding, data uncertainty, and truncation.

Rounding errors, which are an unavoidable consequence of working in finite precision arithmetic, are largely what this book is about. The remainder of this chapter gives basic insight into rounding errors and their effects.

Uncertainty in the data is always a possibility when we are solving practical problems. It may arise in several ways: from errors in measuring physical quantities, from errors in storing the data on the computer (rounding errors),

1.4. Precision Versus Accuracy

The terms accuracy and precision are often confused or used interchangeably, but it is worth making a distinction between them. *Accuracy* refers to the absolute or relative error of an approximate quantity. *Precision* is the accuracy with which the basic arithmetic operations +,-,*,/ are performed, and for floating point arithmetic is measured by the unit roundoff u (see (1.1)). Accuracy and precision are the same for the scalar computation $c = a*b$, but accuracy can be much worse than precision in the solution of a linear system of equations, for example.

It is important to realize that *accuracy is not limited by precision,* at least in theory. This may seem surprising, and may even appear to contradict many of the results in this book. However, arithmetic of a given precision can be used to simulate arithmetic of arbitrarily high precision, as explained in §25.9. (The catch is that such simulation is too expensive to be of practical use for routine computation.) In all our error analyses there is an implicit assumption that the given arithmetic is not being used to simulate arithmetic of a higher precision.

1.5. Backward and Forward Errors

Suppose that an approximation $\widehat{y}$ to $y = f(x)$ is computed in an arithmetic of precision u, where f is a real scalar function of a real scalar variable. How should we measure the "quality" of ?

In most computations we would be happy with a tiny relative error, $E_{\mathrm{rel}}(\widehat{y}) \approx u$, but this cannot always be achieved. Instead of focusing on the relative error of $\widehat{y}$ we can ask "for what set of data have we actually solved our problem?", that is, for what Δx do we have $\widehat{y} = f(x + \Delta x)$? In general, there may be many such Δx, so we should ask for the smallest one. The value of $|\Delta x|$ (or $\min |\Delta x|$), possibly divided by $|x|$, is called the *backward error.* The absolute and relative errors of $\widehat{y}$ are called *forward errors,* to distinguish them from the backward error. Figure 1.1 illustrates these concepts.

The process of bounding the backward error of a computed solution is called *backward error analysis,* and its motivation is twofold. First, it interprets rounding errors as being equivalent to perturbations in the data. The data frequently contains uncertainties due to previous computations or errors committed in storing numbers on the computer. If the backward error is no larger than these uncertainties then the computed solution can hardly be criticized-it may be the solution we are seeking, for all we know. The second attraction of backward error analysis is that it reduces the question of bounding or estimating the forward error to perturbation theory, which for many problems is well understood (and only has to be developed once, for the

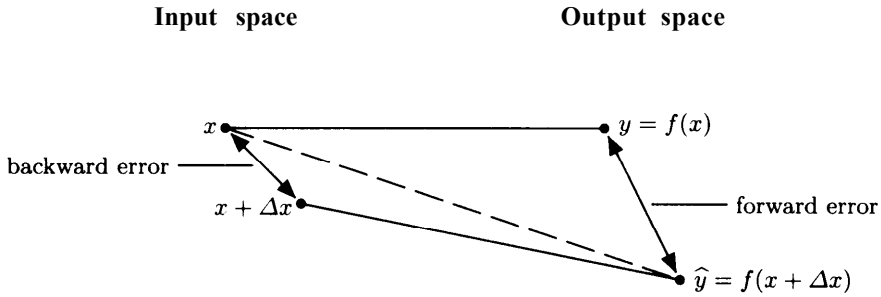

Figure 1.1. *Backward and forward errors for* $y = f(x)$. *Solid line = exact; dotted line = computed.*

given problem, and not for each method). We discuss perturbation theory in the next section.

A method for computing $y = f(x)$ is called *backward stable* if, for any it produces a computed $\hat{y}$ with a small backward error, that is, $\hat{y} = f(x+\Delta x)$ for some small Δx. The definition of "small" will be context dependent. In general, a given problem has several methods of solution, some of which are backward stable and some not.

As an example, assumption (1.1) says that the computed result of the operation $x \pm y$ is the exact result for perturbed data $x(1 + \delta)$ and $y(1 + \delta)$ with $|\delta| \leq u$; thus addition and subtraction are, by assumption, backward stable operations.

Most routines for computing the cosine function do not satisfy $\hat{y} = \cos(x + \Delta x)$ with a relatively small Δx, but only the weaker relation $\hat{y} + \Delta y = \cos(x + \Delta x)$, with relatively small Δy and Δx. A result of the form

$$\hat{y} + \Delta y = f(x + \Delta x), \qquad |\Delta y| \leq \epsilon |y|, \quad |\Delta x| \leq \eta |x| \tag{1.2}$$

is known *as* a *mixed forward-backward* error result and is illustrated in Figure 1.2. Provided that ϵ and η are sufficiently small, (1.2) says that *the computed value* $\hat{y}$ *scarcely differs from the value* $\hat{y} + \Delta y$ *that would have been produced by an input* $x + \Delta x$ *scarcely different from the actual input* x. Even more simply, $\hat{y}$ is almost the right answer for almost the right data.

In general, an algorithm is called *numerically stable* if it is stable in the mixed forward-backward error sense of (1.2) (hence a backward stable algorithm can certainly be called numerically stable). Note that this definition is specific to problems where rounding errors are the dominant form of errors. The term *stability* has different meanings in other areas of numerical analysis.

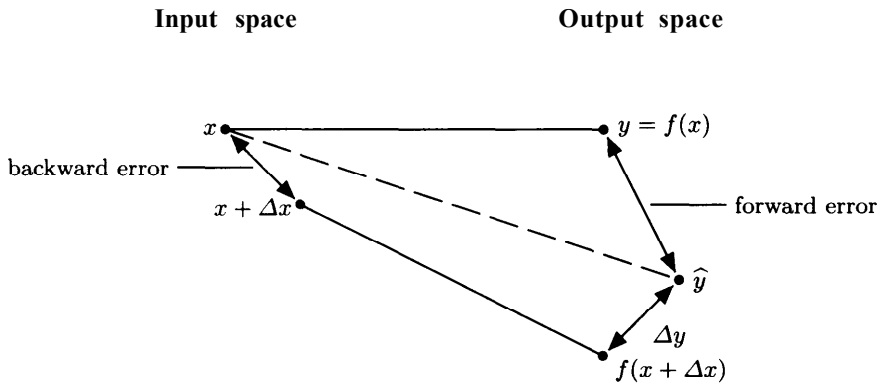

Figure 1.2. *Mixed forward-backward error for $y = f(x)$. Solid line = exact; dotted line = computed.*

1.6. Conditioning

The relationship between forward and backward error for a problem is governed by the conditioning of the problem, that is, the sensitivity of the solution to perturbations in the data. Continuing the $y = f(x)$ example of the previous section, let an approximate solution $\hat{y}$ satisfy $\hat{y} = f(x + \Delta x)$. Then, assuming for simplicity that f is twice continuously differentiable,

$$\hat{y} - y = f(x + \Delta x) - f(x) = f'(x)\Delta x + \frac{f''(x + \theta\Delta x)}{2!}(\Delta x)^2, \quad \theta \in (0, 1),$$

and we can bound or estimate the right-hand side. This expansion leads to the notion of condition number. Since

$$\frac{\hat{y} - y}{y} = \left(\frac{xf'(x)}{f(x)}\right)\frac{\Delta x}{x} + O((\Delta x)^2),$$

the quantity

$$c(x) = \left|\frac{xf'(x)}{f(x)}\right|$$

measures, for small Δx, the relative change in the output for a given relative change in the input, and it is called the *(relative) condition number* of f. If x or f is a vector then the condition number is defined in a similar way using norms and it measures the *maximum* relative change, which is attained for some, but not all, vectors Δx.

As an example, consider the function $f(x) = \log x$. The condition number is $c(x) = |1/\log x|$, which is large for $x \approx 1$. This means that a small relative change in x can produce a much larger relative change in $\log x$ for $x \approx 1$. The

reason is that a small relative change in produces a small *absolute* change in $f(x) = \log x$ (since $f(x+\Delta x) \approx f(x) + f'(x)\Delta x = f(x) + \Delta x/x$) and that change in $\log x$ may be large in a relative sense.

When backward error, forward error, and the condition number are defined in a consistent fashion we have the useful rule of thumb that

$$\text{forward error} \quad \text{condition number} \times \text{backward error},$$

with approximate equality possible. One way to interpret this rule of thumb is to say that *the computed solution to an ill-conditioned problem can have a large forward error.* For even if the computed solution has a small backward error, this error can be amplified by a factor as large as the condition number when passing to the forward error.

One further definition is useful. If a method produces answers with forward errors of similar magnitude to those produced by a backward stable method, then it is called *forward stable.* Such a method need not be backward stable itself. Backward stability implies forward stability, but not vice versa. An example of a method that is forward stable but not backward stable is Cramer's rule for solving a 2 × 2 linear system, which is discussed in §1.10.1.

1.7. Cancellation

Cancellation is what happens when two nearly equal numbers are subtracted. It is often, but not always, a bad thing. Consider the function $f(x) = (1 - \cos x)/x^2$. With $x = 1.2\times10^{-5}$ the value of $\cos x$ rounded to 10 significant figures is

$$c = 0.9999\ 9999\ 99,$$

so that

$$1 - c = 0.0000\ 0000\ 01.$$

Then $(1 - c)/x^2 = 10^{-10}/1.44\times10^{-10} = 0.6944\ldots$, which is clearly wrong given the fact that $0 \le f(x) \le 1/2$ for all $x \ne 0$. A 10 significant figure approximation to $\cos x$ is therefore not sufficient to yield a value of $f(x)$ with even one correct figure. The problem is that $1 - c$ has only 1 significant figure. The subtraction $1 - c$ is *exact,* but this subtraction produces a result of the same size as the error in c. In other words, the subtraction elevates the importance of the earlier error. In this particular example it is easy to rewrite $f(x)$ to avoid the cancellation. Since $\cos x = 1 - 2\sin^2(x/2)$,

$$f(x) = \frac{1}{2}\left(\frac{\sin(x/2)}{x/2}\right)^2.$$

Evaluating this second formula for $f(x)$ with a 10 significant figure approximation to $\sin(x/2)$ yields $f(x) = 0.5$, which is correct to 10 significant figures.

To gain more insight into the cancellation phenomenon consider the subtraction (in exact arithmetic) $\hat{x} = \hat{a} - \hat{b}$, where $\hat{a} = a(1+\Delta a)$ and $\hat{b} = b(1+\Delta b)$. The terms Δa and Δb are relative errors or uncertainties in the data, perhaps attributable to previous computations. With $x = a - b$ we have

$$\left|\frac{x-\hat{x}}{x}\right| = \left|\frac{-a\Delta a - b\Delta b}{a-b}\right| \leq \max(|\Delta a|, |\Delta b|)\frac{|a|+|b|}{|a-b|}.$$

The relative error bound for $\hat{x}$ is large when $|a - b| \ll |a| + |b|$, that is, when there is heavy cancellation in the subtraction. This analysis shows that subtractive cancellation causes relative errors or uncertainties already present in $\hat{a}$ and b to be magnified. In other words, subtractive cancellation *brings earlier errors into prominence.*

It is important to realize that cancellation is not always a bad thing. There are several reasons. First, the numbers being subtracted may be error free, as when they are from initial data that is known exactly. The computation of divided differences, for example, involves many subtractions, but half of them involve the initial data and are harmless for suitable orderings of the points (see §5.3 and §21.3). The second reason is that cancellation may be a symptom of intrinsic ill conditioning of a problem, and may therefore be unavoidable. Third, the effect of cancellation depends on the role that the result plays in the remaining computation. For example, if $x \gg y \approx z > 0$ then the cancellation in the evaluation of $x + (y - z)$ is harmless.

1.8. Solving a Quadratic Equation

Mathematically, the problem of solving the (real) quadratic equation $ax^2 + bx + c = 0$ is trivial: there are two roots (if $a \neq 0$)) given by

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}. \tag{1.3}$$

Numerically, the problem is more challenging, as neither the successful evaluation of (1.3) nor the accuracy of the computed roots can be taken for granted.

The easiest issue to deal with is the choice of formula for computing the roots. If $b^2 \gg |4ac|$ then $\sqrt{b^2 - 4ac} \approx |b|$, and so for one choice of sign the formula (1.3) suffers massive cancellation. This is damaging cancellation because one of the arguments, $(\sqrt{b^2 - 4ac})$, is inexact, so the subtraction brings into prominence the earlier rounding errors. How to avoid the cancellation is well known: obtain the larger root (in absolute value), x_1, from

$$x_1 = \frac{-\left(b + \operatorname{sign}(b)\sqrt{b^2 - 4ac}\right)}{2a},$$

and the other from the equation $x_1 x_2 = c/a$.

Unfortunately, there is a more pernicious source of cancellation: the subtraction $b^2 - 4ac$. Accuracy is lost here when $b^2 \approx 4ac$ (the case of nearly equal roots), and no algebraic rearrangement can avoid the cancellation. The only way to guarantee accurate computed roots is to use double precision (or some trick tantamount to the use of double precision) in the evaluation of $b^2 - 4ac$.

Another potential difficulty is underflow and overflow. If we apply the formula (1.3) in IEEE single precision arithmetic (described in §2.3) to the equation $10^{20}x^2 - 3\cdot 10^{20}x + 2\cdot 10^{20} = 0$ then overflow occurs, since the maximum floating point number is of order 10^{38} the roots, however, are innocuous: $x = 1$ and $x = 2$. Dividing through the equation by $\max(|a|, |b|, |c|) = 10^{20}$ cures the problem, but this strategy is ineffective for the equation $10^{-20}x^2 - 3x + 2\cdot 10^{20} = 0$, whose roots are 10^{20} and $2\cdot 10^{20}$. In the latter equation we need to scale the variable: defining $x = 10^{20}y$ gives $10^{20}y^2 - 3\cdot 10^{20}y + 2\cdot 10^{20} = 0$, which is the first equation we considered. These ideas can be built into a general scaling strategy (see the Notes and References), but the details are nontrivial.

As this discussion indicates, not only is it difficult to devise an accurate and robust algorithm for solving a quadratic equation, but it is a nontrivial task to prepare specifications that define precisely what "accurate" and "robust" mean for a given system of floating point arithmetic.

1.9. Computing the Sample Variance

In statistics the sample variance of n numbers $x_1, \ldots, x_n$ is defined as

$$s_n^2 = \frac{1}{n-1}\sum_{i=1}^{n}(x_i - \bar{x})^2, \tag{1.4}$$

where the sample mean

$$\bar{x} = \frac{1}{n}\sum_{i=1}^{n} x_i.$$

Computing s_n^2 from this formula requires two passes through the data, one to compute $\bar{x}$ and the other to accumulate the sum of squares. A two-pass computation is undesirable for large data sets or when the sample variance is to be computed as the data is generated. An alternative formula, found in many statistics textbooks, uses about the same number of operations but requires only one pass through the data:

$$s_n^2 = \frac{1}{n-1}\left(\sum_{i=1}^{n} x_i^2 - \frac{1}{n}\left(\sum_{i=1}^{n} x_i\right)^2\right). \tag{1.5}$$

This formula is very poor in the presence of rounding errors because it computes the sample variance as the difference of two positive numbers, and therefore can suffer severe cancellation that leaves the computed answer dominated by roundoff. In fact, the computed answer can be negative, an event aptly described by Chan, Golub, and LeVeque [194, 1983] as "a blessing in disguise since this at least alerts the programmer that disastrous cancellation has occurred". In contrast, the original formula (1.4) always yields a very accurate (and nonnegative) answer, unless n is large (see Problem 1.10). Surprisingly, current calculators from more than one manufacturer (but not Hewlett-Packard) appear to use the one-pass formula, and they list it in their manuals.

As an example, if $x = [10000, 10001, 10002]^T$ then, in single precision arithmetic ($u \approx 6 \times 10^{-8}$), the sample variance is computed as 1.0 by the two-pass formula (relative error 0) but 0.0 by the one-pass formula (relative error 1). It might be argued that this data should be shifted by some estimate of the mean before applying the one-pass formula ($x_i \rightarrow x_i - d$, $i = 1\colon n$, which does not change s_n^2), but a good estimate is not always available and there are alternative one-pass algorithms that will always produce an acceptably accurate answer. For example, instead of accumulating $\sum_i x_i$ and $\sum_i x_i^2$ we can accumulate

$$M_k := \frac{1}{k}\sum_{i=1}^{k} x_i \quad \text{and} \quad Q_k := \sum_{i=1}^{k}(x_i - M_k)^2 = \sum_{i=1}^{k} x_i^2 - \frac{1}{k}\left(\sum_{i=1}^{k} x_i\right)^2,$$

which can be done via the updating formulae

$$M_1 = x_1, \qquad M_k = M_{k-1} + \frac{x_k - M_{k-1}}{k}, \qquad k = 2\colon n, \tag{1.6a}$$

$$Q_1 = 0, \qquad Q_k = Q_{k-1} + \frac{(k-1)(x_k - M_{k-1})^2}{k}, \qquad k = 2\colon n, \tag{1.6b}$$

after which $s_n^2 = Q_n/(n-1)$. Note that the only subtractions in these recurrences are relatively harmless ones that involve the data x_i. For the numerical example above, (1.6) produces the exact answer. The updating formulae (1.6) are numerically stable, though their error bound is not as small as the one for the two-pass formula (it is proportional to the condition number K_N in Problem 1.7).

The problem of computing the sample variance illustrates well how mathematically equivalent formulae can have different numerical stability properties.

1.10. Solving Linear Equations

For an approximate solution y to a linear system $Ax = b$ ($A \in \mathbb{R}^{n\times n}$, $b \in \mathbb{R}^n$) the forward error is defined as $\|x-y\|/\|x\|$, for some appropriate norm.

Another measure of the quality of y, more or less important depending on the circumstances, is the size of the residual $r = b - Ay$. When the linear system comes from an interpolation problem, for example, we are probably more interested in how closely Ay represents b than in the accuracy of y. The residual is scale dependent: multiply A and b by a and r is multiplied by a. One way to obtain a scale-independent quantity is to divide by $\|A\|\,\|y\|$, yielding the *relative residual*

$$\rho(y) := \frac{\|b - Ay\|}{\|A\|\,\|y\|}.$$

The importance of the relative residual is explained by the following result, which was probably first proved by Wilkinson (see the Notes and References). We use the 2-norm, defined by $\|x\|_2 = (x^T x)^{1/2}$ and $\|A\|_2 = \max_{x \neq 0} \|Ax\|_2/\|x\|_2$.

Lemma 1.1. *With the notation above, and for the 2-norm,*

$$\rho(y) = \min\left\{ \frac{\|\Delta A\|_2}{\|A\|_2} : (A + \Delta A)y = b \right\}.$$

Proof. If $(A+\Delta A)y = b$ then $r := b-Ay = \Delta Ay$, so $\|r\|_2 < \|\Delta A\|_2 \|y\|_2$, giving

$$\frac{\|\Delta A\|_2}{\|A\|_2} \geq \frac{\|r\|_2}{\|A\|_2\|y\|_2} = \rho(y). \tag{1.7}$$

On the other hand, $(A+\Delta A)y = b$ for $\Delta A = ry^T/(y^T y)$ and $\|\Delta A\|_2 = \|r\|_2/\|y\|_2$, so the bound (1.7) is attainable. □

Lemma 1.1 says that $p(y)$ measures how much A (but not b) must be perturbed in order for y to be the exact solution to the perturbed system, that is, $p(y)$ equals a normwise relative backward error. If the data A and b are uncertain and $p(y)$ is no larger than this uncertainty (e.g., $p(y) = O(u)$) then the approximate solution y must be regarded as very satisfactory. For other problems the backward error may not be as easy to compute as it is for a general linear system, as we will see for the Sylvester equation (§15.2) and the least squares problem (§19.7).

To illustrate these concepts we consider two specific linear equation solvers: Gaussian elimination with partial pivoting (GEPP) and Cramer's rule.

1.10.1. GEPP Versus Cramer's Rule

Cramer's rule says that the components of the solution to a linear system $Ax = b$ are given by $x_i = \det(A_i(b))/\det(A)$, where $A_i(b)$ denotes A with its ith column replaced by b. These formulae are a prime example of a method

that is mathematically elegant, but useless for solving practical problems. The two flaws in Cramer's rule are its computational expense and its numerical instability. The computational expense needs little comment, and is, fortunately, explained in most modern linear algebra textbooks (for example, Strang [961, 1993] cautions the student "it would be crazy to solve equations that way"). The numerical instability is less well known, but not surprising. It is present even for $n = 2$, as a numerical example shows.

We formed a 2×2 system $Ax = b$ with condition number $\kappa_2(A) = \|A\|_2\|A^{-1}\|_2 \approx 10^{13}$, and solved the system by both Cramer's rule and GEPP in MATLAB (unit roundoff $u \approx 1.1 \times 10^{-16}$). The results were as follows, where $r = b - A\widehat{x}$:

Cramer's rule		GEPP	
$\widehat{x}$	$r/(\|A\|_2\|\widehat{x}\|_2)$	$\widehat{x}$	$r/(\|A\|_2\|\widehat{x}\|_2)$
1.0000	1.5075×10^{-7}	1.0002	-4.5689×10^{-17}
2.0001	1.9285×10^{-7}	2.0004	-2.1931×10^{-17}

The scaled residual for GEPP is pleasantly small-of order the unit roundoff. That for Cramer's rule is ten orders of magnitude larger, showing that the computed solution $\widehat{x}$ from Cramer's rule does not closely satisfy the equations, or, equivalently, does not solve a nearby system. The solutions themselves are similar, both being accurate to three significant figures in each component but incorrect in the fourth significant figure. This is the accuracy we would expect from GEPP because of the rule of thumb "forward error $\lesssim$ backward error $\times$ condition number". That Cramer's rule is as accurate as GEPP in this example, despite its large residual, is perhaps surprising, but it is explained by the fact that Cramer's rule is forward stable for $n = 2$; see Problem 1.9. For general n, the accuracy and stability of Cramer's rule depend on the method used to evaluate the determinants, and satisfactory bounds are not known even for the case where the determinants are evaluated by GEPP.

The small residual produced by GEPP in this example is typical: error analysis shows that GEPP is guaranteed to produce a relative residual of order u when $n = 2$ (see §9.2). To see how remarkable a property this is, consider the rounded version of the exact solution: $z = fl(x) = x + \Delta x$, where $\|\Delta x\|_2 \le u\|x\|_2$. The residual of z satisfies $\|b - Az\|_2 = \|-A\Delta x\|_2 \le u\|A\|_2\|x\|_2 \approx u\|A\|_2\|z\|_2$. Thus *the computed solution from GEPP has about as small a residual as the rounded exact solution, irrespective of its accuracy.*

Expressed another way, the errors in GEPP are highly correlated so as to produce a small residual. To emphasize this point, the vector [1.0006,2.0012], which agrees with the exact solution of the above problem to five significant figures (and therefore is more accurate than the solution produced by GEPP), has a relative residual $\|r\|_2/(\|A\|_2\|\widehat{x}\|_2)$ of order 10^{-6}.

Table 1.1. *Computed approximations* $\widehat{f}_n = fl((1+1/n)^n)$ *to* $e = 2.71828\ldots$

n	$\widehat{f}_n$	$\lvert e - \widehat{f}_n \rvert$
10^1	2.593743	1.25×10^{-1}
10^2	2.704811	1.35×10^{-2}
10^3	2.717051	1.23×10^{-3}
10^4	2.718597	3.15×10^{-4}
10^5	2.721962	3.68×10^{-3}
10^6	2.595227	1.23×10^{-1}
10^7	3.293968	5.76×10^{-1}

1.11. Accumulation of Rounding Errors

Since the first electronic computers were developed in the 1940s, comments along the following lines have often been made: "The enormous speed of current machines means that in a typical problem many millions of floating point operations are performed. This in turn means that rounding errors can potentially accumulate in a disastrous way." This sentiment is true, but misleading. Most often, instability is caused not by the accumulation of millions of rounding errors. but by the insidious growth of just a few rounding errors.

As an example, let us approximate $e = \exp(1)$ by taking finite n in the definition $e := \lim_{n\to\infty}(1 + 1/n)^n$. Table 1.1 gives results computed in Fortran 90 in single precision ($u \approx 6 \times 10^{-8}$).

The approximations are poor, degrading as n approaches the reciprocal of the machine precision. For n a power of 10, l/n has a nonterminating binary expansion. When $1+1/n$ is formed for n a large power of 10, only a few significant digits from l/n are retained in the sum. The subsequent exponentiation to the power n, even if done exactly, must produce an inaccurate approximation to e(indeed, doing the exponentiation in double precision does not change any of the numbers shown in Table 1.1). Therefore a single rounding error is responsible for the poor results in Table 1.1.

There is a way to compute $(1+1/n)^n$ more accurately, using only single precision arithmetic; it is the subject of Problem 1.5.

Strassen's method for fast matrix multiplication provides another example of the unpredictable relation between the number of arithmetic operations and the error. If we evaluate $fl(AB)$ by Strassen's method, for $n\times n$ matrices A and B, and we look at the error as a function of the recursion threshold $n_0 \le n$. we find that while the number of operations decreases as n_0 decreases from n to 8, the error typically *increases;* see §22.2.2.

1.12. Instability Without Cancellation

It is tempting to assume that calculations free from subtractive cancellation must be accurate and stable, especially if they involve only a small number of operations. The three examples in this section show the fallacy of this assumption.

1.12.1. The Need for Pivoting

Suppose we wish to compute an LU factorization

$$A = \begin{bmatrix} \epsilon & -1 \\ 1 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ l_{21} & 1 \end{bmatrix} \begin{bmatrix} u_{11} & u_{12} \\ 0 & u_{22} \end{bmatrix}, \qquad 0 < \epsilon \ll 1.$$

Clearly, $u_{11} = \epsilon$, $u_{12} = -1$, $l_{21} = \epsilon^{-1}$, and $u_{22} = 1 - l_{21}u_{12} = 1 + \epsilon^{-1}$. In floating point arithmetic, if ϵ is sufficiently small then $\hat{u}_{22} = fl(1 + \epsilon^{-1})$ evaluates to ϵ^{-1}. Assuming l_{21} is computed exactly, we then have

$$A - \hat{L}\hat{U} = \begin{bmatrix} \epsilon & -1 \\ 1 & 1 \end{bmatrix} - \begin{bmatrix} 1 & 0 \\ \epsilon^{-1} & 1 \end{bmatrix} \begin{bmatrix} \epsilon & -1 \\ 0 & \epsilon^{-1} \end{bmatrix} = \begin{bmatrix} 0 & 0 \\ 0 & 1 \end{bmatrix}.$$

Thus the computed LU factors fail completely to-reproduce A. Notice that there is no subtraction in the formation of L and U. Furthermore, the matrix A is very well conditioned $(\kappa_\infty(A) = 4/(1+\epsilon))$. The problem, of course, is with the choice of ϵ as the pivot. The partial pivoting strategy would interchange the two rows of A before factorizing it, resulting in a stable factorization.

1.12.2. An Innocuous Calculation?

For any $x \geq 0$ the following computation leaves x unchanged:

```
for i = 1:60
    x = √x
end
for i = 1:60
    x = x^2
end
```

Since the computation involves no subtractions and all the intermediate numbers lie between 1 and x, we might expect it to return an accurate approximation to x in floating point arithmetic.

On the HP 48G calculator, starting with $x = 100$ the algorithm produces $x = 1.0$. In fact, for any x, the calculator computes, in place of $f(x) = x$, the function

$$\hat{f}(x) = \begin{cases} 0, & 0 \leq x < 1, \\ 1, & x \geq 1. \end{cases}$$

The calculator is producing a completely inaccurate approximation to $f(x)$ in just 120 operations on nonnegative numbers. How can this happen?

The positive numbers x representable on the HP 48G satisfy $10^{-499} \le x \le 9.999\ldots \times 10^{499}$. If we define $r(x) = x^{1/2^{60}}$ then, for any machine number $x \ge 1$,

$$\begin{aligned} 1 \le r(x) < r(10^{500}) &= 10^{500/2^{60}} \\ &= e^{500 \cdot 2^{-60} \cdot \log 10} < e^{10^{-15}} \\ &= 1 + 10^{-15} + \tfrac{1}{2} \cdot 10^{-30} + \cdots, \end{aligned}$$

which rounds to 1, since the HP 48G works to about 12 decimal digits. Thus for $x > 1$, the repeated square roots reduce x to 1.0, which the squarings leave unchanged.

For $0 < x < 1$ we have

$$x \le 0.\underbrace{99\ldots9}_{12}$$

on a 12-digit calculator, so we would expect the square root to satisfy

$$\begin{aligned} \sqrt{x} \le (1 - 10^{-12})^{1/2} &= 1 - \tfrac{1}{2} \cdot 10^{-12} - \tfrac{1}{8} \cdot 10^{-24} - \cdots \\ &= 0.\underbrace{99\ldots9}_{12}4\underbrace{99\ldots9}_{11}87499\ldots \end{aligned}$$

This upper bound rounds to the 12 significant digit number 0.99. . .9. Hence after the 60 square roots we have on the calculator a number $x \le 0.99\ldots9$. The 60 squarings are represented by $s(x) = x^{2^{60}}$, and

$$\begin{aligned} s(x) \le s(0.99\ldots9) &= (1 - 10^{-12})^{2^{60}} \\ &= 10^{2^{60} \log(1-10^{-12}) \log_{10} e} \\ &\approx 10^{-2^{60} \cdot 10^{-12} \cdot \log_{10} e} \\ &\approx 3.6 \times 10^{-500708}. \end{aligned}$$

Because it is smaller than the smallest positive representable number, this result is set to zero on the calculator--a process known as *underflow*. (The converse situation, in which a result exceeds the largest representable number, is called *overflow.)*

The conclusion is that there is nothing wrong with the calculator. This innocuous-looking calculation simply exhausts the precision and range of a machine with 12 digits of precision and a 3-digit exponent.

1.12.3. An Infinite Sum

It is well known that $\sum_{k=1}^{\infty} k^{-2} = \pi^2/6 = 1.6449\ 3406\ 6848\ldots$. Suppose we were not aware of this identity and wished to approximate the sum numerically. The most obvious strategy is to evaluate the sum for increasing k until

the computed sum does not change. In Fortran 90 in single precision this yields the value 1.6447 2532, which is first attained at $k = 4096$. This agrees with the exact infinite sum to just four significant digits out of a possible nine.

The explanation for the poor accuracy is that we are summing the numbers from largest to smallest, and the small numbers are unable to contribute to the sum. For $k = 4096$ we are forming $s + 4096^{-2} = s + 2^{-24}$, where $s \approx 1.6$. Single precision corresponds to a 24-bit mantissa, so the term we are adding to s "drops off the end" of the computer word, as do all successive terms.

The simplest cure for this inaccuracy is to sum in the opposite order: from smallest to largest. Unfortunately, this requires knowledge of how many terms to take before the summation begins. With 10^9 terms we obtain the computed sum 1.6449 3406, which is correct to eight significant digits.

For much more on summation, see Chapter 4.

1.13. Increasing the Precision

When the only source of errors is rounding, a common technique for estimating the accuracy of an answer is to recompute it at a higher precision and to see how many digits of the original and the (presumably) more accurate answer agree. We would intuitively expect any desired accuracy to be achievable by computing at a high enough precision. This is certainly the case for algorithms possessing an error bound proportional to the precision, which includes all the algorithms described in the subsequent chapters of this book. However, since an error bound is not necessarily attained, there is no guarantee that a result computed in t digit precision will be more accurate than one computed in s digit precision, for a given $t > s$; in particular, for a very ill conditioned problem both results could have no correct digits.

For illustration, consider the system $Ax = b$, where A is the inverse of the 5×5 Hilbert matrix and $b_i = (-1)^i i$. (For details of the matrices used in this experiment see Chapter 26.) We solved the system in varying precisions with unit roundoffs $u = 2^{-t}$, $t = 15{:}40$, corresponding to about 4 to 12 decimal places of accuracy. (This was accomplished in MATLAB by using the function chop from the Test Matrix Toolbox to round the result of every arithmetic operation to t bits; see Appendix E.) The algorithm used was Gaussian elimination (without pivoting), which is perfectly stable for this symmetric positive definite matrix. The upper plot of Figure 1.3 shows t against the relative errors $\|x - \widehat{x}\|_\infty / \|x\|_\infty$ and the relative residuals $\|b - A\widehat{x}\|_\infty / (\|A\|_\infty \|\widehat{x}\|_\infty)$. The lower plot of Figure 1.3 gives corresponding results for $A = P_5 + 5I$, where P_5 is the Pascal matrix of order 5. The condition numbers $\kappa_\infty(A)$ are 1.62×10^2 for the inverse Hilbert matrix and 9.55×10^5 for the shifted Pascal matrix. In both cases the general trend is that increasing the precision decreases the residual and relative error, but the behaviour is

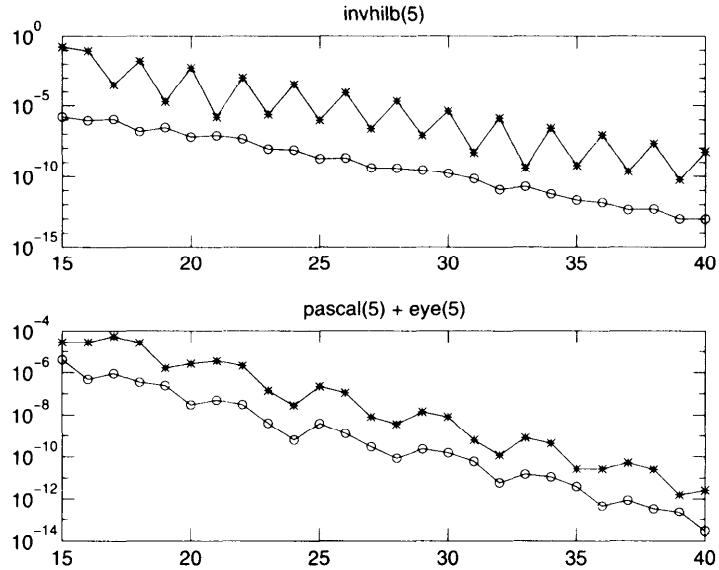

Figure 1.3. *Forward errors* $\|x - \widehat{x}\|_\infty / \|x\|_\infty$ ("$*$") *and relative residuals* $\|b - A\widehat{x}\|_\infty / (\|A\|_\infty \|\widehat{x}\|_\infty)$ ("o") *versus precision* $t = -\log_2 u$ *on the x axis.*

not monotonic. The reason for the pronounced oscillating behaviour of the relative error (but not the residual) for the inverse Hilbert matrix is not clear.

An example in which increasing the precision by several bits does not improve the accuracy is the evaluation of

$$y = x + a\,\sin(bx), \qquad x = \frac{1}{7}, \quad a = 10^{-8}, \quad b = 2^{24}. \tag{1.8}$$

Figure 1.4 plots t versus the absolute error, for precisions $u = 2^{-t}$, $t = 10{:}40$. Since $a\,\sin(bx) \approx -8.55\times10^{-9}$, for t less than about 20 the error is dominated by the error in representing $x = 1/7$. For $22 \le t \le 31$ the accuracy is (exactly) constant! The plateau over the range $22 \le t \le 31$ is caused by a fortuitous rounding error in the addition: in the binary representation of the exact answer the 23rd to 32nd digits are 1s, and in the range of t of interest the final rounding produces a number with a 1 in the 22nd bit and zeros beyond, yielding an unexpectedly small error that affects only bits 33rd onwards.

A more contrived example in which increasing the precision has no beneficial effect on the accuracy is the following evaluation of $z = f(x)$:

```
y = abs(3(x-0.5)-0.5)/25
if y = 0
   z = 1
```

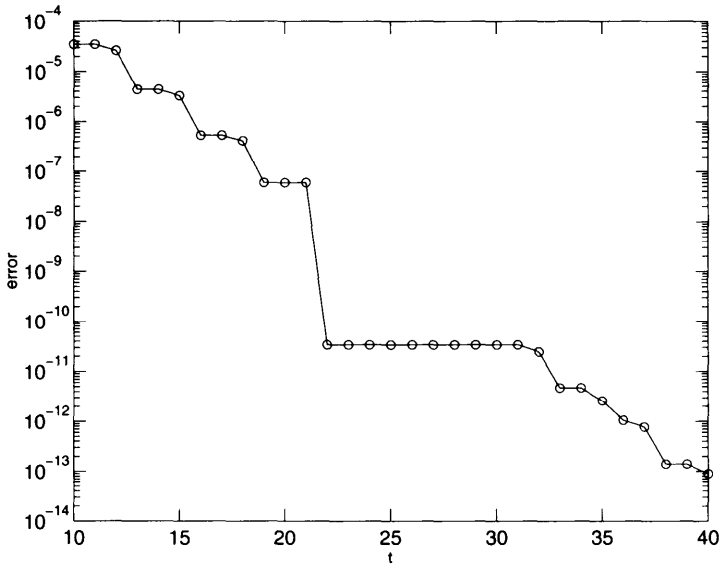

Figure 1.4. *Absolute error versus precision,* $t = -\log_2 u$.

```
else
   z = e^y    % Store to inhibit extended precision evaluation.
   z = (z - 1)/y
end
```

In exact arithmetic, $z = f(2/3) = 1$, but in Fortran 90 on a Sun SPARCstation and on a 486DX workstation, $\hat{z} = fl(f(2/3)) = 0.0$ in both single and double precision arithmetic. A further example is provided by the "innocuous calculation" of §1.12.2, in which a step function is computed in place of $f(x) = x$ for a wide range of precisions.

It is worth stressing that *how* precision is increased can greatly affect the results obtained. Increasing the precision without preserving important properties such as monotonicity of rounding can vitiate an otherwise reliable algorithm. Increasing the precision without maintaining a correct relationship among the precisions in different parts of an algorithm can also be harmful to the accuracy.

1.14. Cancellation of Rounding Errors

It is not unusual for rounding errors to cancel in stable algorithms, with the result that the final computed answer is much more accurate than the inter-

mediate quantities. This phenomenon is not universally appreciated, perhaps because we tend to look at the intermediate numbers in an algorithm only when something is wrong, not when the computed answer is sat isfactory. We describe two examples. The first is a very short and rather unusual computation, while the second involves a well-known algorithm for computing a standard matrix decomposition.

1.14.1. Computing $(e^x - 1)/x$

Consider the function $f(x) = (e^x - 1)/x = \sum_{i=0}^{\infty} x^i/(i+1)!$, which arises in various applications. The obvious way to evaluate f is via the algorithm

```
% Algorithm 1.
if x = 0
  f = 1
else
  f = (e^x - 1)/x
end
```

This algorithm suffers severe cancellation for $|x| << 1$, causing it to produce an inaccurate answer (0 instead of 1, if x is small enough) . Here is an alternative:

```
% Algorithm 2.
y = e^x
if y = 1
  f = 1
else
  f = (y - 1)/logy
end
```

At first sight this algorithm seems perverse, since it evaluates both exp and log instead of just exp. Some results computed in MATLAB are shown in Table 1.2. All the results for Algorithm 2 are correct in all the significant figures shown, except for $x = 10^{-15}$, when the last digit should be 1. On the other hand, Algorithm 1 returns answers that become less and less accurate as x decreases.

To gain insight we look at the numbers in a particular computation with $x = 9 \times 10^{-8}$ and $u = 2^{-24} \approx 6 \times 10^{-8}$, for which the correct answer is 1.00000005 to the significant digits shown. For Algorithm 1 we obtain a completely inaccurate result, as expected:

$$fl\left(\frac{e^x - 1}{x}\right) \equiv fl\left(\frac{1.19209290 \times 10^{-7}}{9.00000000 \times 10^{-8}}\right) = 1.32454766.$$

Table 1.2. *Computed values of* $(e^x - 1)/x$ *from Algorithms* 1 *and* 2.

x	Algorithm 1	Algorithm 2
10^{-5}	1.000005000006965	1.000005000016667
10^{-6}	1.000000499962184	1.000000500000167
10^{-7}	1.000000049433680	1.000000050000002
10^{-8}	$9.999999939225290 \times 10^{-1}$	1.000000005000000
10^{-9}	1.000000082740371	1.000000000500000
10^{-10}	1.000000082740371	1.000000000050000
10^{-11}	1.000000082740371	1.000000000005000
10^{-12}	1.000088900582341	1.000000000000500
10^{-13}	$9.992007221626408 \times 10^{-1}$	1.000000000000050
10^{-14}	$9.992007221626408 \times 10^{-1}$	1.000000000000005
10^{-15}	1.110223024625156	1.000000000000000
10^{-16}	0	

Algorithm 2 produces a result correct in all but the last digit:

$$fl\left(\frac{e^x - 1}{\log e^x}\right) \equiv fl\left(\frac{1.19209290 \times 10^{-7}}{1.19209282 \times 10^{-7}}\right) = 1.00000006.$$

Here are the quantities that would be obtained by Algorithm 2 in exact arithmetic (correct to the significant digits shown):

$$\frac{e^x - 1}{\log e^x} \equiv \frac{9.00000041 \times 10^{-8}}{9.00000001 \times 10^{-8}} = 1.00000005.$$

We see that Algorithm 2 obtains very inaccurate values of $e^x - 1$ and $\log e^x$, but the ratio of the two quantities it computes is very accurate. Conclusion: errors cancel in the division in Algorithm 2.

A short error analysis explains this striking cancellation of errors. We assume that the exp and log functions are both computed with a relative error not exceeding the unit roundoff u. The algorithm first computes $\widehat{y} = e^x(1+\delta)$, $|\delta| \leq u$. If $\widehat{y} = 1$ then $e^x(1+\delta) = 1$, so

$$x = -\log(1+\delta) = -\delta+\delta^2/2-\delta^3/3+\ldots, \qquad |\delta| \leq u,$$

which implies that the correctly rounded value of $f(x) = 1+x/2+x^2/6+\ldots$ is 1, and so f has been evaluated correctly, to the working precision. If $\widehat{y} \neq 1$ then[7], using (1.1),

$$\widehat{f} = fl\big((\widehat{y} - 1)/\log \widehat{y}\big) = \frac{(\widehat{y} - 1)(1 + \epsilon_1)}{\log \widehat{y}\,(1 + \epsilon_2)}(1 + \epsilon_3), \tag{1-9}$$

[7]The analysis from this point on assumes the use of a guard digit in subtraction (see §2.4); without a guard digit Algorithm 2 is not highly accurate.

Obsidian

where $|\epsilon_i| \leq u$, $i = 1{:}3$. Defining $v = \widehat{y} - 1$, we have

$$\begin{aligned} g(\widehat{y}) := \frac{\widehat{y}-1}{\log \widehat{y}} &= \frac{v}{\log(1+v)} \\ &= \frac{v}{v - v^2/2 + v^3/3 - \cdots} = \frac{1}{1 - v/2 + v^2/3 - \cdots} \\ &= 1 + \frac{v}{2} + O(v^2). \end{aligned}$$

For small x ($y \approx 1$)

$$g(\widehat{y}) - g(y) \approx \frac{\widehat{y}-y}{2} \approx \frac{e^x \delta}{2} \approx \frac{\delta}{2} \approx \frac{g(y)\delta}{2}.$$

From (1.9) it follows that $\widehat{f}$ approximates f with relative error at most about $3.5u$.

The details of the analysis obscure the crucial property that ensures its success. For small x, neither $\widehat{y} - 1$ nor $\log \widehat{y}$ agrees with its exact arithmetic counterpart to high accuracy. But $(\widehat{y} - 1)/\log \widehat{y}$ is an extremely good approximation to $(y\text{-}1)/\log y$ near $y=1$, because the function $g(y) = (y\text{-}1)/\log y$ varies so slowly there (g has a removable singularity at 1 and $g'(1) = 1$). In other words, the errors in $\widehat{y} - 1$ and $\log \widehat{y}$ almost completely cancel.

1.14.2. QR Factorization

Εννοεί το R

Any matrix $A \in \mathbb{R}^{m \times n}$, $m \geq n$, has a QR factorization $A = QR$, where $Q \in \mathbb{R}^{m \times}$ has orthonormal columns and $R \in \mathbb{R}^{n \times x}$ is upper trapezoidal ($r_{ij} = 0$ for $i > j$). One way of computing the QR factorization is to premultiply A by a sequence of Givens rotations-orthogonal matrices G that differ from the identity matrix only in a 2×2 principal submatrix, which has the form

$$\begin{bmatrix} \cos\theta & \sin\theta \\ -\sin\theta & \cos\theta \end{bmatrix}.$$

With $A_1 := A$, a sequence of matrices A_k satisfying $A_k = G_k A_{k-1}$ is generated. Each A_k, has one more zero than the last, so $A_p = R$ for $p = n(m - (n + 1)/2)$. To be specific, we will assume that the zeros are introduced in the order $(n,1)$. $(n - 1,1)$, . . . , $(2,1)$; $(n,2)$, . . . , $(3,2)$; and so on.

For a particular 10×6 matrix A, Figure 1.5 plots the relative errors $\|A_k - A_k\|_2/\|A\|_2$, where A_k, denotes the matrix computed in single precision arithmetic ($u \approx 6 \times 10^{-8}$). We see that many of the intermediate matrices are very inaccurate, but the final computed $\widehat{R}$ has an acceptably small relative error, of order u. Clearly, there is heavy cancellation of errors on the last few stages of the computation. This matrix $A \in \mathbb{R}^{10 \times 6}$ was specially chosen,

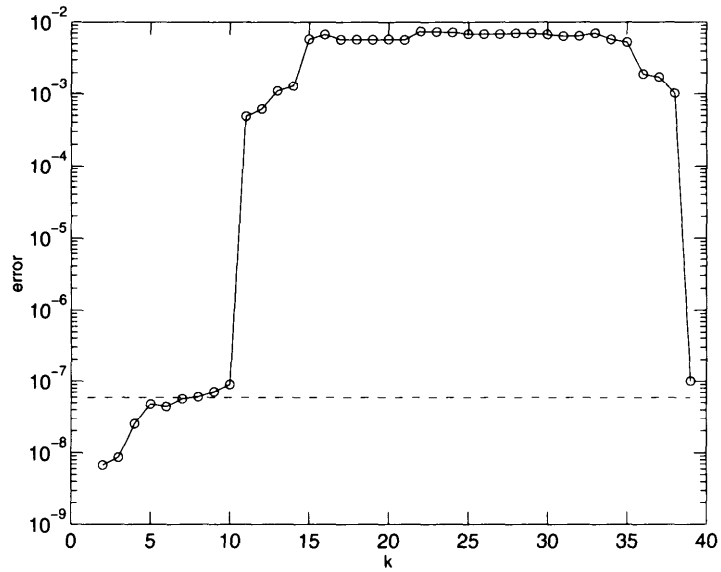

Figure 1.5. *Relative errors* $\|A_k - \hat{A}_k\|_2/\|A\|_2$ *for Givens QR factorization. The dotted line is the unit roundoff level.*

following a suggestion of Wilkinson [1100, 1985], as a full matrix such that $\|A\|_2 \approx 1$ and A_{10} has the form

$$A_{10} = \begin{bmatrix} \tilde{a}_{11} & \tilde{a}_{12} & \tilde{A}(1,3{:}n) \\ 0 & 1 & \tilde{A}(2,3{:}n) \\ 0 & y & \tilde{A}(3{:}m,3{:}n) \end{bmatrix},$$

Because y is at the roundoff level, the computed $\hat{y}$ is the result of severe subtractive cancellation and so is dominated by rounding errors. Consequently, the computed Givens rotations $\tilde{G}_{10}, \ldots, \tilde{G}_{17}$, whose purpose is to zero the vector $\hat{y}$, and which are determined by ratios involving the elements of $\hat{y}$, bear little relation to their exact counterparts, causing $\hat{A}_k$ to differ greatly from A_k for $k = 11,12,\ldots$.

To shed further light on this behaviour, we note that the Givens QR factorization is perfectly backward stable; that is, the computed R is the exact R factor of $A + \Delta A$, where $\|\Delta A\|_2 < cu\|A\|_2$, with c a modest constant depending on the dimensions (Theorem 18.9). By invoking a perturbation result for the QR factorization (namely (18.27)) we conclude that $\|R - \hat{R}\|_2/\|A\|_2$ is bounded by a multiple of $\kappa_2(A)u$. Our example is constructed so that $\kappa_2(A)$ is small (≈ 24), so we know a priori that the graph in Figure 1.5 must eventually dip down to the unit roundoff level.

We also note that $\|Q - \widehat{Q}\|_2$ is of order u in this example, as again we can show it must be from perturbation theory. Since Q is a product of Givens rotations, this means that even though some of the intermediate Givens rotations are very inaccurate, their product is highly accurate, so in the formation of Q, too, there is extensive cancellation of rounding errors.

1.15. Rounding Errors Can Be Beneficial

An old method for computing the largest eigenvalue (in absolute value) of a matrix A and the corresponding eigenvector is the power method, which consists of repeatedly multiplying a given starting vector by A. With scaling to avoid underflow and overflow, the process in its simplest form is

```
% Choose a starting vector x.
while not converged
      x := Ax
      x := x/||x||∞
end
```

The theory says that if A has a unique eigenvalue of largest modulus and x is not deficient in the direction of the corresponding eigenvector υ, then the power method converges to a multiple of υ (at a linear rate).

Consider the matrix

$$A = \begin{bmatrix} 0.4 & -0.6 & 0.2 \\ -0.3 & 0.7 & -0.4 \\ -0.1 & -0.4 & 0.5 \end{bmatrix},$$

which has eigenvalues 0, 0.4394 and 1.161 (correct to the digits shown) and an eigenvector $[1, 1, 1]^T$ corresponding to the eigenvalue zero. If we take $[1,1,1]^T$ as the starting vector for the power method then, in principle, the zero vector is produced in one step, and we obtain no indication of the desired dominant eigenvalue-eigenvector pair. However, when we carry out the computation in MATLAB, the first step produces a vector with elements of order 10^{-16} and we obtain after 38 iterations a good approximation to the dominant eigenpair. The explanation is that the matrix A cannot be stored exactly in binary floating point arithmetic. The computer actually works with $A + \Delta A$ for a tiny perturbation ΔA, and the dominant eigenvalue and eigenvector of $A + \Delta A$ are very good approximations to those of A. The starting vector $[1,1,1]^T$ contains a nonzero (though tiny) component of the dominant eigenvector of $A + \Delta A$. This component grows rapidly under multiplication by $A + \Delta A$, helped by rounding errors in the multiplication, until convergence to the dominant eigenvector is obtained.

Perhaps an even more striking example of beneficial effects of rounding errors is in inverse iteration, which is just the power method applied to the shifted and inverted matrix $(A-\mu I)^{-1}$. The shift μ is usually an approximate eigenvalue. The closer μ is to an eigenvalue, the more nearly singular $A - \mu I$ is, and hence the larger the error in computing $y = (A - \mu I)^{-1} x$ (which is done by solving $(A-\mu I)y = x$). However, it can be shown that the error lies almost entirely in the direction of the required eigenvector, and so is harmless; see, for example, Parlett [820, 1980, §4.31] or Golub and Van Loan [470, 1989, §7.6.1].

1.16. Stability of an Algorithm Depends on the Problem

An algorithm can be stable as a means for solving one problem but unstable when applied to another problem. One example is the modified Gram-Schmidt method, which is stable when used to solve the least squares problem but can give poor results when used to compute an orthonormal basis of a matrix (see §§18.7 and 19.3).

A lesser known and much simpler example is Gaussian elimination (GE) without pivoting for computing the determinant of an upper Hessenberg matrix. A square matrix A is upper Hessenberg if $a_{ij} = 0$ for $i > j + 1$. GE transforms A to upper triangular form by $n - 1$ row eliminations. one for each of the boxed entries in this 4×4 illustration:

$$A = \begin{bmatrix} \times & \times & \times & \times \\ \boxed{\times} & \times & \times & \times \\ 0 & \boxed{\times} & \times & \times \\ 0 & 0 & \boxed{\times} & \times \end{bmatrix} \longrightarrow \begin{bmatrix} \times & \times & \times & \times \\ 0 & \times & \times & \times \\ 0 & 0 & \times & \times \\ 0 & 0 & 0 & \times \end{bmatrix} = U.$$

The determinant of A is given by the product of the diagonal elements of U. It is easy to show that this is a stable way to evaluate $\det(A)$, even though arbitrarily large multipliers may arise during the elimination. Note. first, that, if $A^{(k)}$ denotes the matrix at the start of the kth stage ($A^{(1)} = A$), then

$$u_{kk} = a_{kk}^{(k)} = a_{kk}^{(k-1)} - \frac{a_{k,k-1}^{(k-1)} a_{k-1,k}^{(k-1)}}{a_{k-1,k-1}^{(k-1)}} = a_{kk} - \frac{a_{k,k-1} a_{k-1,k}^{(k-1)}}{a_{k-1,k-1}^{(k-1)}},$$

because the kth row of $A^{(k-1)}$ is the same as the kth row of A. In floating point arithmetic the model (1.1) shows that the computed $\hat{a}_{ij}^{(k)}$ satisfy

$$\begin{aligned} \hat{u}_{kk} = \hat{a}_{kk}^{(k)} &= \left(a_{kk} - \frac{a_{k,k-1} \hat{a}_{k-1,k}^{(k-1)}}{\hat{a}_{k-1,k-1}^{(k-1)}} (1+\epsilon_1^{(k)})(1+\epsilon_2^{(k)}) \right) (1+\epsilon_3^{(k)}) \\ &= a_{kk}(1+\epsilon_3^{(k)}) - \frac{[a_{k,k-1}(1+\epsilon_1^{(k)})(1+\epsilon_2^{(k)})(1+\epsilon_3^{(k)})] \hat{a}_{k-1,k}^{(k-1)}}{\hat{a}_{k-1,k-1}^{(k-1)}}, \end{aligned}$$

Table 1.3. *Results from GE without pivoting on an upper Hessenberg matrix.*

	Exact	Computed	Relative error
x:	$\begin{bmatrix} 1.0000 \\ 1.0000 \\ 1.0000 \\ 1.0000 \end{bmatrix}$	$\begin{bmatrix} 2.3842 \\ 1.0000 \\ 1.0000 \\ 1.0000 \end{bmatrix}$	1.3842
$\det(A)$:	2.0000	2.0000	1.9209×10^{-8}

where $|\epsilon_i^{(k)}| \leq u$, $i = 1{:}3$. This equation says that the computed diagonal elements $\widehat{u}_{kk}$ are the exact diagonal elements corresponding not to A, but to a matrix obtained from A by changing the diagonal elements to $a_{kk}(1 + \epsilon_3^{(k)})$ and the subdiagonal elements to $a_{k,k-1}(1 + \epsilon_1^{(k)})(1 + \epsilon_2^{(k)})(1 + \epsilon_3^{(k)})$. In other words, the computed $\widehat{u}_{kk}$ are exact for a matrix differing negligibly from A. The computed determinant d, which is given by

$$\widehat{d} = fl(\widehat{u}_{11} \dots \widehat{u}_{nn}) = \widehat{u}_{11} \dots \widehat{u}_{nn}(1 + \eta_1) \dots (1 + \eta_n), \qquad |\eta_i| \leq u,$$

is therefore a tiny relative perturbation of the determinant of a matrix differing negligibly from A. so this method for evaluating $\det(A)$ is numerically stable (in the mixed forward backward error sense of (1.2)).

However, if we use GE without pivoting to solve an upper Hessenberg linear system then large multipliers can cause the solution process to be unstable. If we try to extend the analysis above we find that the computed LU factors (as opposed to just the diagonal of U) do not, as a whole, necessarily correspond to a small perturbation of A.

A numerical example illustrates these ideas. Let

$$A = \begin{bmatrix} \alpha & -1 & -1 & -1 \\ 1 & 1 & -1 & -1 \\ 0 & 1 & 1 & -1 \\ 0 & 0 & 1 & 1 \end{bmatrix}.$$

We took $a = 10^{-7}$ and $b = Ae$($e = [1,1,1,1]^T$) and used GE without pivoting in single precision arithmetic ($u \approx 6 \times 10^{-8}$) to solve $Ax = b$ and compute $\det(A)$. The computed and exact answers are shown to five significant figures in Table 1.3. Not surprisingly, the computed determinant is very accurate. But the computed solution to $Ax = b$ has no correct figures in its first component. This reflects instability of the algorithm rather than ill conditioning of the problem because the condition number $\kappa_\infty(A) = 16$. The source of the instability is the large first multiplier, $a_{21}/a_{11} = 10^7$.

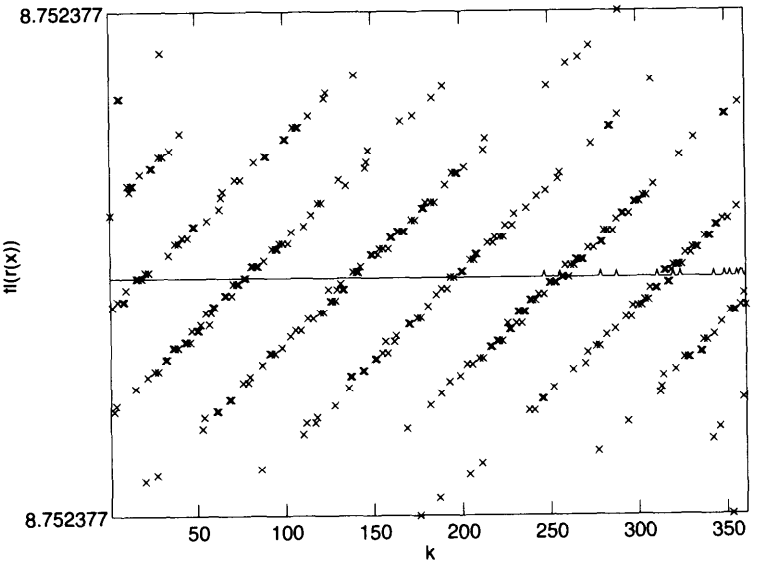

Figure 1.6. *Values* of *rational function r(x) computed by Horner's rule (marked as "×"), for* $x = 1.606 + (k - 1)2^{-52}$; *solid line is the "exact" r(x).*

1.17. Rounding Errors Are Not Random

Rounding errors, and their accumulated effect on a computation, are not random. This fact underlies the success of many computations, including some of those described earlier in this chapter. The validity of statistical analysis of rounding errors is discussed in §2.6. Here we simply give a revealing numerical example (due to W. Kahan).

Define the rational function

$$r(x) = \frac{622 - x(751 - x(324 - x(59 - 4x)))}{112 - x(151 - x(72 - x(14 - x)))},$$

which is expressed in a form corresponding to evaluation of the quartic polynomials in the numerator and denominator by Horner's rule. We evaluated $r(x)$ by Horner's rule in double precision arithmetic for 361 consecutive floating point numbers starting with $a = 1.606$, namely $x = a + (k - 1)2^{-52}$, $k = 1{:}361$; the function $r(x)$ is virtually constant on this interval. Figure 1.6 plots the computed function values together with a much more accurate approximation to $r(x)$ (computed from a continued fraction representation). The striking pattern formed by the values computed by Horner's rule shows clearly that the rounding errors in this example are not random.

1.18. Designing Stable Algorithms

There is no simple recipe for designing numerically stable algorithms. While this helps to keep numerical analysts in business (even in proving each other's algorithms to be unstable!) it is not good news for computational scientists in general. The best advice is to be *aware* of the need for numerical stability when designing an algorithm and not to concentrate solely on other issues, such as computational cost and parallelizability.

A few guidelines can be given.

1. Try to avoid subtracting quantities contaminated by error (though such subtractions may be unavoidable).

2. Minimize the size of intermediate quantities relative to the final solution. The reason is that if intermediate quantities are very large then the final answer may be the result of damaging subtractive cancellation. Looked at another way, large intermediate numbers swamp the initial data, resulting in loss of information. The classic example of an algorithm where this considerat ion is import ant is Gaussian elimination (§9.2), but an even simpler one is recursive summation (§4.2).

3. Look for different formulations of a computation that are mathematically but not numerically equivalent. For example, the classical Gram-Schmidt method is unstable, but a trivial modification produces the stable modified Gram-Schmidt (MGS) method (§18.7). There are two ways of using the MGS method to solve a least squares problem, the more obvious of which is unstable (§19.3).

4. It is advantageous to express update formulae as

   $$\text{new-value} = \text{old-value} + \text{small-correct ion}$$

   if the small correction can be computed with many correct significant figures. Numerical methods are often naturally expressed in this form; examples include met hods for solving ordinary differential equations, where the correction is proportional to a step size, and Newton's method for solving a nonlinear system. A classic example of the use of this update strategy is in iterative refinement for improving the computed solution to a linear system $Ax = b$, in which by computing residuals $r = b - Ay$ in extended precision and solving update equations that have the residuals as right-hand sides a highly accurate solution can be computed: see Chapter 11. For another example (in which the correction is not necessarily small), see Problem 2.8.

5. Use only well-conditioned transformations of the problem. In matrix computations this amounts to multiplying by orthogonal matrices where

possible, instead of nonorthogonal, and possibly, ill-conditioned matrices. See §6.2 for a simple explanation of this advice in terms of norms.

6. Take precautions to avoid unnecessary overflow and underflow (see §25.8).

Concerning the second point, good advice is to look at the numbers generated during a computation. This was common practice in the early days of electronic computing. On some machines it was unavoidable because the contents of the store were displayed on lights or monitor tubes! Wilkinson gained much insight into numerical stability by inspecting the progress of an algorithm, and sometimes altering its course (for an iterative process with parameters): "Speaking for myself I gained a great deal of experience from user participation, and it was this that led to my own conversion to backward error analysis" [1099, 1980, pp. 112-113] see also [1083, 1955]). It is ironic that with the wealth of facilities we now have for tracking the progress of numerical algorithms (multiple windows in colour, graphical tools, fast printers) we often glean less than Wilkinson and his co-workers did from mere paper tape and lights.

1.19. Misconceptions

Several common misconceptions and myths have been dispelled in this chapter (none of them for the first time-see the Notes and References). We highlight them in the following list.

1. Cancellation in the subtraction of two nearly equal numbers is always a bad thing (§1.7).

2. Rounding errors can overwhelm a computation only if vast numbers of them accumulate (§1.11).

3. A short computation free from cancellation, underflow, and overflow must be accurate (§1.12).

4. Increasing the precision at which a computation is performed increases the accuracy of the answer (§1.13).

5. The final computed answer from an algorithm cannot be more accurate than any of the intermediate quantities, that is, errors cannot cancel (§1.14).

6. Rounding errors can only hinder, not help, the success of a computation (§1.15).

1.20. Rounding Errors in Numerical Analysis

Inevitably, much of this book is concerned with numerical linear algebra, because this is the area of numerical analysis in which the effects of rounding errors are most important and have been most studied. In nonlinear problems rounding errors are often not a major concern because other forms of error dominate. Although we give examples for numerical methods involving derivatives and integrals (for example, Euler's met hod in §4.3 and quadrature in Problem 3.12), it is beyond our scope to give a treatment of the effects and influence of rounding errors on these methods. We do. however, give some selected references to the literature. grouped by subject area.

Approximation theory: Clenshaw [212, 19551], Cox [249, 19721], [250, 19751], [251, 1978], Cox and Harris [253, 1989], de Boor [272, 19721], and de Boor and Pinkus [273, 1977].

Chaos and dynamical systems: Cipra [211, 1988], Coomes, Koçak, and Palmer [242, 1995], Corless [246. 1992], [247, 1992], Hammel, Yorke, and Grebogi [499, 1988], and Sanz-Serna and Larsson [893, 1993].

Nonlinear equations: Dennis and Walker [302, 1984], Spellucci [933, 1980], and Wozniakowski [1111, 1977].

Optimization: Dennis and Schnabel [300, 1983], Fletcher [377, 1986], [379, 1988], [380, 1993], [381, 1994]. Gill, Murray, and Wright [447, 1981], Gurwitz [490, 1992], Müller-Merbach [783, 1970], and Wolfe [1108, 1965].

Ordinary differential equation initial value problems: Henrici [517, 1962], [518, 1963], [519, 1964], D. J. Higham [525, 1991]. Sanz-Serna [891, 1992, §12], Sanz-Serna and Calvo [892, 1994], and Shampine [910, 1994, §3.3, §5.6].

Partial differential equations: Ames [14, 1977], Birkhoff and Lynch [101, 1984], Canuto, Hussaini, Quarteroni, and Zang [183, 1988], Douglas [319, 1959], Forsythe and Wasow [397, 1960], Richtmyer and Morton [872, 1967, §1.8], and Trefethen and Trummer [1020, 1987].

Quadrature: Davis and Rabinowitz [267, 1984, §4.2], Lyness [717, 1969], and Piessens et al. [832, 1983, §3.4.3.1].

1.21. Notes and References

The term "correct significant digits" is rarely defined in textbooks: it is apparently assumed that the definition is obvious. One of the earliest books on

numerical analysis, by Scarborough [897, 1950] (first edition 1930), is noteworthy for containing theorems describing the relationship between correct significant digits and relative error.

The first definition of correct significant digits in §1.2 is suggested by Hildebrand [571, 1974, §1.41], who notes its weaknesses.

For a formal proof and further explanation of the fact that precision does not limit accuracy see Priest [844, 1992].

It is possible to develop formal definitions of numerical stability, either with respect to a particular problem, as is frequently done in research papers, or for a very general class of problems, as is done, for example, by de Jong [274, 1977]. Except in §7.6, we do not give formal definitions of stability in this book, preferring instead to adapt informally the basic notions of backward and forward stability to each problem, and thereby to minimize the amount of notation and abstraction.

Backward error analysis was systematically developed, exploited, and popularized by Wilkinson in the 1950s and 1960s in his research papers and, in particular, through his books [1088, 1963], [1089, 1965] (for more about the books see the Notes and References for Chapter 2). Backward error ideas had earlier appeared implicitly in papers by von Neumann and Goldstine (1057, 1947] and Turing [1027, 1948], both of which deal with the solution of linear systems, and explicitly in an unpublished technical report of Givens [451, 1954] on the solution of the symmetric eigenproblem by reduction to tridiagonal form followed by the use of Sturm sequences. The concept of backward error is not limited to numerical linear algebra. It is used, for example, in the numerical solution of differential equations; see Eirola [349, 1993], Enright [352, 1989], and Shampine [910, 1994, §2.2], in addition to the references of Sanz-Serna and his co-authors cited in §1.20. Backward error is also used in understanding chaotic behaviour of iterations; see the references in §1.20.

Conditioning of problems has been studied by numerical analysts since the 1940s but the first general theory was developed by Rice [871, 1966]. In numerical linear algebra, developing condition numbers is part of the subject of perturbation theory, on which there is a large literature.

The solution of a quadratic equation is a classic problem in numerical analysis. In 1969 Forsythe [393, 1969] pointed out "the near absence of algorithms to solve even a quadratic equation in a satisfactory way on actually used digital computer systems" and he presented specifications suggested by Kahan for a satisfactory solver. Similar, but less technical, presentations are given by Forsythe [392, 1969] and Forsythe, Malcolm, and Moler [395, 1977, §2.6]. Kahan [627, 1972] and Sterbenz [938, 1974] both present algorithms for solving a quadratic equation, accompanied by error analysis.

For more details of algorithms for computing the sample variance and their error analysis, see Chan and Lewis [195, 1979], Chan, Golub, and LeVeque [194, 1983], Barlow [61, 1991], and the references therein. Good general

references on computational aspects of statistics are Kennedy and Gentle [649, 1980] and Thisted [1000. 1988].

The issues of conditioning and numerical stability play a role in any discipline in which finite precision computation is performed, but the understanding of these issues is less well developed in some disciplines than in others. In geometric computation, for example, there has been much interest since the late 1980s in the accuracy and robustness of geometric algorithms; see Milenkovic [754, 1988], Hoffmann [576, 1989], and Priest [843, 1991], [844, 1992].

It was after discovering Lemma 1.1 that Wilkinson began to develop backward error analysis systematically in the 1950s. He explains that in solving eigenproblems $Ax = \lambda x$ by deflation, the residual of the computed solution, $r := A\overline{x} - \overline{\lambda}\overline{x}$ (with the normalization $\overline{x}^T\overline{x} = 1$) was "always at noise level relative to A" [1101, 1986]. He continues, "After some years' experience of this I happened. almost by accident, to observe that . . . $(A - r\overline{x}^T)\overline{x} = \overline{\lambda}\overline{x}$ . . . In other words $\overline{\lambda}$ and $\overline{x}$ were exact for a matrix $A - r\overline{x}^T$ and since $\|r\overline{x}^T\|_2 = \|r\|_2$, this meant that they were exact for a matrix differing from A at the noise level of the computer." For further details see [1101, 1986] or [1100, 1985].

The numerical stability of Cramer's rule for 2×2 systems has been investigated by Moler [768, 1974] and Stummel [964, 1981, §3.3].

The example in §1.12.2 is taken from the *HP-15C Advanced Functions Handbook* [523, 1982], and a similar example is given by Kahan [629, 1980]. For another approach to analysing this "innocuous calculation" see Problem 3.11. The "$f(2/3)$" example in §1.13 is also taken from [629, 1980], in which Kahan states three "anti-theorems" that are included among our misconceptions in §1.19.

The example (1.8) is adapted from an example of Sterbenz [938, 1974, p. 220], who devotes a section to discussing the effects of rerunning a computation at higher precision.

The function expm1 := e^x - 1 is provided in some floating point processors and mathematics libraries as a more accurate alternative to forming e^x and subtracting 1 [991, 1992]. It is important in the computation of sinh and tanh, for example (since $\sinh x = e^{-x}(e^{2x} - 1)/2$). Algorithm 2 in §1.14.1 is due to Kahan [629, 1980].

The instability. and stability of GE without pivoting applied to an upper Hessenberg matrix (§1.16) was first pointed out and explained by Wilkinson [1084, 1960]; Parlett [818, 1965] also gives a lucid discussion. In the 1950s and 1960s. prior to the development of the QR algorithm, various methods were proposed for the nonsymmetric eigenvalue problem that involved transforming a matrix to Hessenberg form H and then finding the zeros of the characteristic polynomial $\det(H - \lambda I)$. The most successful method of this type was Laguerre's iteration. described by Parlett [817, 1964], and used in conjunction with Hyman's method for evaluating $\det(H - \lambda I)$. Hyman's met hod

is described in §13.5.1.

Classic papers dispensing good advice on the dangers inherent in numerical computation are the "pitfalls" papers by Stegun and Abramowitz [937, 1956] and Forsythe [394, 1970]. The book *Numerical Methods That Work* by Acton [4, 1970] must also be mentioned as a fount of hard-earned practical advice on numerical computation (look carefully and you will see that the front cover includes a faint image of the word "Usually" before "Work"). If it is not obvious to you that the equation $x^2 - 10x + 1 = 0$ is best thought of as a nearly linear equation for the smaller root, you will benefit from reading Acton (see p. 58). Everyone should read Acton's "Interlude: What *Not* To Compute" (pp. 245-257).

Finally, we mention the paper "How to Get Meaningless Answers in Scientific Computation (and What to Do About it)" by Fox [401, 1971]. Fox, a contemporary of Wilkinson, founded the Oxford Computing Laboratory and was for many years Professor of Numerical Analysis at Oxford. In this paper he gives numerous examples in which incorrect answers are obtained from plausible numerical methods (many of the examples involve truncation errors as well as rounding errors). The section titles provide a list of reasons why you might compute worthless answers:

- Your problem might be ill conditioned.
- Your method might be unstable.
- You expect too much "analysis" from the computers.
- Your intuition fails you.
- You accept consistency too easily.
- A successful method may fail in slightly different circumstances.
- Your test examples may be too special.

Fox estimates [401, 1971, p. 296] that "about 80 per cent of all the results printed from the computer are in error to a much greater extent than the user would believe."

[8] This reason refers to using an inappropriate convergence test in an iterative process.

Problems

The road to wisdom?
Well, it's plain and simple to express:
Err
and err
and err again
but less
and less
and less.

-PIET HEIN, *Grooks (1966)*

1.1. In error analysis it is sometimes convenient to bound $\widetilde{E}_{\rm rel}(\widehat{x}) = |x-\widehat{x}|/|\widehat{x}|$ instead of $E_{\rm rel}(\widehat{x}) = |x - \widehat{x}|/|x|$. Obtain inequalities between $E_{\rm rel}(\widehat{x})$ and $\widetilde{E}_{\rm rel}(\widehat{x})$.

1.2. (Skeel and Keiper [923, 1993, §1.2]) The number $y = e^{\pi\sqrt{163}}$ was evaluated at t digit precision for several values of t, yielding the values shown in the following table. which are in error in at most one unit in the least significant digit (the first two values are padded with trailing zeros):

t	y
10	2625374 12600000000
15	262537412640769000
20	262537412640768744.00
25	262537412640768744.0000000
30	262537412640768743.999999999999

Does it follow that the last digit before the decimal point is 4?

1.3. Show how to rewrite the following expressions to avoid cancellation for the indicated arguments.

1. $\sqrt{x+1} - 1$, $x \approx 0$.
2. $\sin x - \sin y$, $x \approx y$.
3. $x^2 - y^2$, $x \approx y$.
4. $(1 - \cos x)/\sin x$, $x \approx 0$.
5. $c = (a^2 + b^2 - 2ab\cos\theta)^{1/2}$, $a \approx b$, $|\theta| \ll 1$.

1.4. Give stable formulae for computing the square root $x + iy$ of a complex number $a + ib$.

1.5. [523, 1982] By writing $(1 + 1/n)^n = \exp(n\log(1 + 1/n))$, show how to compute $(1 + 1/n)^n$ accurately for large n. Assume that the log function is computed with a relative error not exceeding u. (Hint: adapt the technique used in §1.14.1.)

1.6. (Smith [928, 1975]) Type the following numbers into your pocket calculator, and look at them upside down (you or the calculator):

07734	The famous "_world" program
38079	Object
318808	Name
35007	Adjective
$57738.57734 \times 10^{40}$	Exclamation on finding a bug
3331	A high quality floating point arithmetic
$\sqrt{31,438,449}$	Fallen tree trunks

1.7. A condition number for the sample variance (1.4), here denoted by $V(x)$: $\mathbb{R}^n \rightarrow \mathbb{R}$, can be defined by

$$\kappa_C := \lim_{\epsilon \rightarrow 0} \sup \left\{ \frac{|V(x) - V(x + \Delta x)|}{\epsilon V(x)} : |\Delta x_i| \leq \epsilon |x_i|,\ i = 1\colon n \right\}.$$

Show that

$$\kappa_C = 2 \frac{\sum_{i=1}^n |x_i - \overline{x}||x_i|}{(n-1)V(x)}.$$

This condition number measures perturbations in x componentwise. A corresponding normwise condition number is

$$\kappa_N := \lim_{\epsilon \rightarrow 0} \sup \left\{ \frac{|V(x) - V(x + \Delta x)|}{\epsilon V(x)} : \|\Delta x\|_2 \leq \epsilon \|x\|_2 \right\}.$$

Show that

$$\kappa_N = 2 \frac{\|x\|_2}{\sqrt{(n-1)V(x)}} = 2 \left(1 + \frac{n}{n-1} \frac{\overline{x}^2}{V(x)} \right)^{1/2} \geq \kappa_C.$$

1.8. (Kahan, Muller, [781, 1989], Francois and Muller [406, 1991]) Consider the recurrence

$$x_{k+1} = 111 - (1130 - 3000/x_{k-1})/x_k, \qquad x_0 = 11/2, \quad x_1 = 61/11.$$

In exact arithmetic the x_k form a monotonically increasing sequence that converges to 6. Implement the recurrence on your computer or pocket calculator and compare the computed x_{34} with the true value 5.998 (to four correct significant figures). Explain what you see.

The following questions require knowledge of material from later chapters.

1.9. Cramer's rule solves a 2×2 system $Ax = b$ according to

$$\begin{aligned} d &= a_{11}a_{22} - a_{21}a_{12}, \\ x_1 &= (b_1 a_{22} - b_2 a_{12})/d, \\ x_2 &= (a_{11}b_2 - a_{21}b_1)/d. \end{aligned}$$

Show that, assuming d is computed exactly (this assumption has little effect on the final bounds), the computed solution $\widehat{x}$ satisfies

$$\frac{\|x - \widehat{x}\|_\infty}{\|x\|_\infty} \leq \gamma_3 \operatorname{cond}(A, x), \qquad \|b - A\widehat{x}\|_\infty \leq \gamma_3 \operatorname{cond}(A^{-1})\|b\|_\infty,$$

where $\gamma_3 = 3u/(1\text{-}3u)$, $\operatorname{cond}(A,x) = \| \, |A^{-1}||A||x| \, \|_\infty / \|x\|_\infty$ and $\operatorname{cond}(A) = \| \, |A^{-1}||A| \, \|_\infty$. This forward error bound is as small as that for a backward stable method (see §7.2, §7.6) so Cramer's rule is forward stable for 2×2 systems.

1.10. Show that the computed sample variance $\widehat{V} = fl(V(x))$ produced by the two-pass formula (1.4) satisfies

$$\frac{|V - \widehat{V}|}{V} \leq (n + 3)u + O(u^2).$$

(Note that this error bound does not involve the condition numbers κ_C or κ_N from Problem 1.7, at least in the first-order term. This is a rare instance of an algorithm that determines the answer more accurately than the data warrants!)